Date __/__/____

☆ Transaction Control Language (TCL):- A transaction is a group of SQL statements executed as a single unit. TCL commands are used to manage transactions in SQL.

(*) Commands :-

1. Start transaction :- Used to begin a new transaction, changes are temporary, until Commit or Rollback is executed.
   Syntax : START TRANSACTION;

2. Commit :- Saves changes permanently in database.
   Syntax : COMMIT;

3. Rollback :- Undo changes before commit.
   Syntax : ROLLBACT TO SAVEPOINT;

4. SAVEPOINT :- Creates temporary checkpoints inside transaction.
   Syntax : SAVEPOINT savepoint_name;

☆ Data Control Language (DCL):- DCL commands control user access and permissions to the database. They are used by DBAs to grant or revoke privileges, ensuring security & restricting unauthorized access.

(*) Commands :-

1. GRANT :- To give access.
   Syntax : GRANT privileges ON table_name TO user;
   :-(Specific access to particular table).

Syntax: GRANT ALL PRIVILEGES ON database_name.* TO user;
--(Access to all table data in a DB)

2. Revoke :- To take back the permission given.
Syntax: REVOKE PRIVILEGES ON table_name FROM user;

☆ WINDOW Functions :-(RANK Function)
Window function is used to assign ranks to rows based on a specified column.
Rows with same value receive same rank.

(*) SQL provides mainly 3 ranking functions :-
1. RANK(). 2. DENSE_RANK(). 3. ROW_NUMBER()
These are window functions used with OVER() clause.

1. RANK() ⇒ Assigns ranks to rows. Duplicate values get the same rank, and next rank is skipped.
Syntax: SELECT column1, column2,
RANK() OVER(ORDER BY column2 DESC) AS rnk
FROM table_name;

2. DENSE_RANK() ⇒ Assigns same rank for duplicates, but does NOT skip ranks.
Syntax: SELECT column1, column2,
DENSE_RANK() OVER(ORDER BY column2 DESC) rank
FROM table_name;

3. ROW_NUMBER() ⇒ Assigns unique row numbers to every row. Duplicates also get different numbers.

Syntax: SELECT column1, column2,
ROW_NUMBER() OVER(ORDER BY column2
DESC) rank.
FROM table_name;

Functions	Duplicate Ranks	Skips Rank
RANK()	Yes	Yes
DENSE_RANK()	Yes	No
ROW_NUMBER()	No	No.

ranking functions cannot be used in WHERE clause directly, instead use subquery.

(*) PARTITION BY clause:- It is used with window function to divide rows into groups and perform calculations separately within each group.
It works like GROUP BY for window functions, but does not reduce rows.

```
SELECT column_name,
    function_name() OVER (
        PARTITION BY column_name
        ORDER BY column_name
    )
FROM table_name;
```

GROUP BY	PARTITION BY
> Combines rows	> Keeps all rows
> Returns one row per group	> Returns all rows
> Used with aggregate functions.	> Used with window functions.

(*) Real - Time Uses :-
> Department - wise salary ranking.
> Monthly sales analysis.
> leaderboard systems.
> Top performers per Team.

★ Stored Procedure :- It is a reusable, precompiled collection of SQL statements stored inside the database to perform specific tasks efficiently.

(*) It helps to ⇒
> Reuse SQL code.
> Improve performance.
> Reduce duplication.

(*) Advantages of Stored ~~performa~~ procedures :-
> Reusability.
> Better performance.
> Security.
> Reduces network traffic.
> Easier Maintenance.

Syntax :
```
DELIMITER //
CREATE PROCEDURE procedure_name ()
BEGIN
SELECT * FROM emp;
END //
DELIMETER;
```

DELIMETER : It is used in MYSQL to change the default statement termination symbol temporarily.

Date ___/___/_____

(*) Types of Parameter :-

Parameter	Purpose
IN	Input value
OUT	Returns Output
INOUT	Both input & Output.

> Syntax: DELIMETER //
CREATE PROCEDURE employees()
BEGIN
SELECT * FROM emp;
END //
DELIMETER;
CALL employees();

> IN Parameter : DELIMETER //
CREATE PROCEDURE employees (IN eid INT)
BEGIN
SELECT *
FROM emp
WHERE empno = eid;
END //
DELIMETER;
CALL employees (7501);

> OUT Parameter : ~~CREA~~ DELIMETER //
CREATE PROCEDURE totalemp (OUT total INT)
BEGIN
SELECT COUNT(*) INTO total
FROM emp;
END //
DELIMETER;
CALL totalemp (@total); - @total is a session variable.

Page No.

> INOUT Parameter : DELIMETER //

```
CREATE PROCEDURE Salary (INOUT Sal
        DECIMAL (10,2))
BEGIN
  SET Sal = Sal + 2000;
  END //
DELIMETER ;
SET @Sal = 3000;
CALL Salary (@ Sal);
SELECT @ Sal;
```

```
DROP PROCEDURE employees;
SHOW PROCEDURE STATUS ;
SHOW ~~create~~ CREATE PROCEDURE employees ;
```

* TRIGGERS :- A trigger is a special type of stored program in MySQL that automatically executes when a event occurs on a table.
Events are : INSERT , UPDATE , DELETE.

(*) Types of Triggers ⇒

Trigger Type	Description.
BEFORE INSERT	Executes before inserting data
AFTER INSERT	Executes after inserting data
BEFORE UPDATE	Executes before updating data
AFTER UPDATE	Executes after updating data
BEFORE DELETE	Executes before deleting
AFTER DELETE	Executes after deleting

Date __/__/____

Syntax: CREATE
DELIMETER //
CREATE TRIGGER trigger_name
BEFORE/AFTER INSERT/DELETE/UPDATE
ON table_name
FOR EACH ROW
BEGIN
-- SQL statements
END //
DELIMETER;

*) SHOW → SHOW TRIGGERS;
*) DROP → DROP TRIGGER trigger_name;

(*) Advantages of Triggers:
> Automatic execution.
> Maintains Audit Logs.
> Enforces business rules.
> Reduces manual work.
> Improves data integrity.

(*) Disadvantages of Triggers:
> Hard to debug.
> Too many triggers reduce performance.
> Hidden execution may confuse developers.

* ACID Properties :- ACID properties are rules that ensure reliable & consistent transaction processing in databases.
ACID stands for:
A → Atomicity ⇒ A transaction is treated as a single unit - either all operations happen or

Page No.

none happen. (all or nothing).

C → Consistency ⇒ A transaction must move database from one valid state to another valid state. (maintains database validity).

I → Isolation ⇒ A transaction should always transaction occuring in same database without any conflicts. (prevents transaction conflicts).

D → Durability ⇒ Once committed, data remains permanently saved even after system crash (ensures committed data is permanent).